Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация с использованием кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26(1)

Пасевич К.Ю.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE).

Ход работы

1. Модифицировать программу из ЛР №1:

- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.

2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:

- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.

4. Провести сравнительный анализ:

    - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
        - MSE + фиксированный шаг (данные из ЛР №1);
        - MSE + адаптивный шаг (данные из ЛР №2);
        - BCE + фиксированный шаг;
        - BCE + адаптивный шаг.
    - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).

5. Визуализировать результаты:

    - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
    - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
    - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.

6. Реализовать режим функционирования:

    - Пользователь вводит координаты x1,x2.

- Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
- Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).

7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Вариант 9

x_1	x_2	e
2	1	0
-2	1	1
2	-1	0
-2	-1	0

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

# обучающая выборка
X = np.array([
    [2, 1],
    [-2, 1],
    [2, -1],
    [-2, -1]
], dtype=float)

y = np.array([0, 1, 0, 0], dtype=float)

epochs = 5000
alpha = 0.01
eps = 0.01


# сигмоида
def sigmoid(z):
    return 1/(1+np.exp(-z))


# BCE
def bce(pred, y):
    e = 1e-9
    return -(y*np.log(pred+e)+(1-y)*np.log(1-pred+e))
```

```
# ========================
# MSE фиксированный шаг
# ========================
def mse_fixed(X,y):

    w=np.random.randn(2)
    b=np.random.randn()

    errors=[]

    for epoch in range(epochs):

        total=0

        for xi,yi in zip(X,y):

            z=np.dot(w,xi)+b
            pred=z

            err=yi-pred
            total+=err**2

            w+=alpha*err*xi
            b+=alpha*err

        errors.append(total/len(X))

        if errors[-1] <= eps:
            break

    return w,b,errors


# ========================
# MSE адаптивный шаг
# ========================
def mse_adaptive(X,y):

    w=np.random.randn(2)
    b=np.random.randn()

    errors=[]
    t=1

    for epoch in range(epochs):

        total=0

        for xi,yi in zip(X,y):

            lr=1/t
            t+=1

            z=np.dot(w,xi)+b
            pred=z

            err=yi-pred
            total+=err**2
```

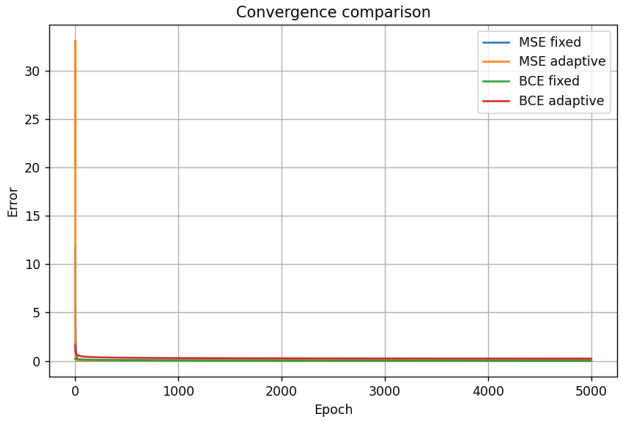
```
            w+=lr*err*xi
            b+=lr*err

        errors.append(total/len(X))

        if errors[-1]<=eps:
            break

    return w,b,errors


# ========================
# BCE фиксированный шаг
# ========================
def bce_fixed(X,y):

    w=np.random.randn(2)
    b=np.random.randn()

    errors=[]

    for epoch in range(epochs):

        total=0

        for xi,yi in zip(X,y):

            z=np.dot(w,xi)+b
            pred=sigmoid(z)

            total+=bce(pred,yi)

            grad=(pred-yi)

            w-=alpha*grad*xi
            b-=alpha*grad

        errors.append(total/len(X))

        if errors[-1]<=eps:
            break

    return w,b,errors


# ========================
# BCE адаптивный шаг
# ========================
def bce_adaptive(X,y):

    w=np.random.randn(2)
    b=np.random.randn()

    errors=[]
    t=1

    for epoch in range(epochs):

        total=0
```

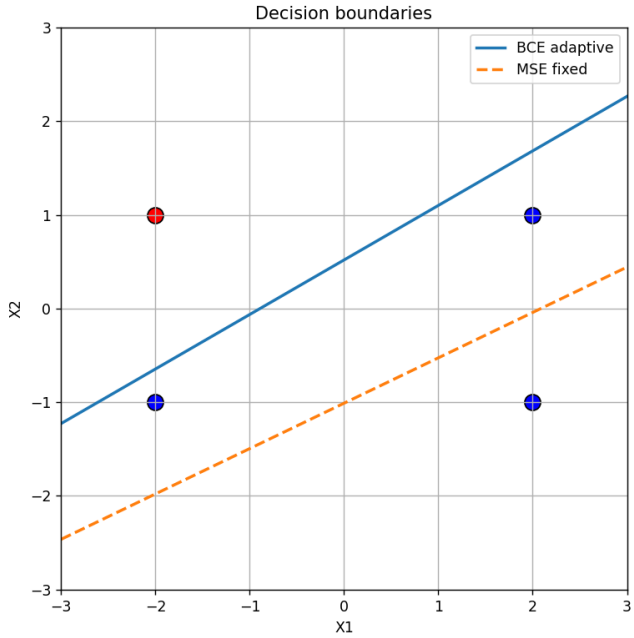
```
        for xi,yi in zip(X,y):

            lr=1/t
            t+=1

            z=np.dot(w,xi)+b
            pred=sigmoid(z)

            total+=bce(pred,yi)

            grad=(pred-yi)

            w-=lr*grad*xi
            b-=lr*grad

        errors.append(total/len(X))

        if errors[-1]<=eps:
            break

    return w,b,errors


# обучение моделей
w1,b1,e1=mse_fixed(X,y)
w2,b2,e2=mse_adaptive(X,y)
w3,b3,e3=bce_fixed(X,y)
w4,b4,e4=bce_adaptive(X,y)


# ========================
# график ошибок
# ========================
plt.figure(figsize=(8,5))

plt.plot(e1,label="MSE fixed")
plt.plot(e2,label="MSE adaptive")
plt.plot(e3,label="BCE fixed")
plt.plot(e4,label="BCE adaptive")

plt.xlabel("Epoch")
plt.ylabel("Error")
plt.title("Convergence comparison")

plt.legend()
plt.grid()

plt.show()


# ========================
# визуализация точек
# ========================
plt.figure(figsize=(7,7))

for i in range(len(X)):

    color="red" if y[i]==1 else "blue"
```

```
    plt.scatter(X[i,0],X[i,1],color=color,s=120,edgecolors="black")


x=np.linspace(-3,3,300)

# линия BCE adaptive
if w4[1]!=0:
    yline=-(w4[0]*x+b4)/w4[1]
    plt.plot(x,yline,label="BCE adaptive",linewidth=2)

# линия MSE fixed
if w1[1]!=0:
    yline2=-(w1[0]*x+b1)/w1[1]
    plt.plot(x,yline2,"--",label="MSE fixed",linewidth=2)

plt.xlim(-3,3)
plt.ylim(-3,3)

plt.grid()
plt.legend()

plt.title("Decision boundaries")
plt.xlabel("X1")
plt.ylabel("X2")

plt.show()


# ========================
# режим функционирования
# ========================
def predict(x1,x2):

    z=w4[0]*x1+w4[1]*x2+b4
    p=sigmoid(z)

    cls=1 if p>=0.5 else 0

    print("Probability:",round(p,4))
    print("Class:",cls)

    plt.figure(figsize=(7,7))

    for i in range(len(X)):
        color="red" if y[i]==1 else "blue"
        plt.scatter(X[i,0],X[i,1],color=color,s=120,edgecolors="black")

    if w4[1]!=0:
        yline=-(w4[0]*x+b4)/w4[1]
        plt.plot(x,yline)

    plt.scatter(x1,x2,color="magenta",s=200,marker="X")

    plt.xlim(-3,3)
    plt.ylim(-3,3)

    plt.grid()
    plt.show()
```

```
while True:

    v1=input("X1 (exit): ")

    if v1.lower()=="exit":
        break

    v2=input("X2: ")

    try:
        predict(float(v1),float(v2))
    except:
        print("Input error")
```

Результат программы:





Вывод: изучила принципы бинарной классификации с вероятностной точки зрения. Реализовала однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE).